Workshop No. 4 - Containerization, Acceptance, and CI/CD

Software Engineering Seminar

Universidad Distrital Francisco José de Caldas

Systems Engineering Program

Presented by:

Brayan Alejandro Riveros Rodríguez
20201020084

Carlos Andrés Pescador Castro
20182020139

Cristian Santiago Ríos Vásquez
20201020107

Bogotá D.C., Colombia

November 29, 2025

Contents

1 Containerization

1.1 Dockerfiles

For this project, each backend includes a custom Dockerfile. The goal of these Dockerfiles is to produce lightweight and efficient container images ready for deployment in local and production environments.

1.1.1 Java backend

The Java backend Dockerfile uses a multi-stage build. In the first stage, Maven downloads all dependencies and creates the application JAR file. In the second stage, a smaller Amazon Corretto runtime image is used to run the final application. This approach reduces the final image size and improves performance. The container exposes port 8080 and runs the service using a simple java -jar command.

Listing 1: Dockerfile for Java Backend

```
    # === Build Stage ===
FROM maven:3.9.11-amazoncorretto-25-alpine AS build

WORKDIR /app

# Copy and prepare dependency configuration
COPY pom.xml .

# Download and cache dependencies
RUN mvn dependency:go-offline -B

# Copy source code
COPY src ./src

# Build project while skipping tests
RUN mvn clean package -DskipTests

# === Runtime Stage ===
FROM amazoncorretto:25-alpine AS runtime

WORKDIR /app

# Copy the packaged JAR file
COPY --from=build /app/target/*.jar app.jar

# Expose service port
EXPOSE 8080

# Start the application
ENTRYPOINT ["java", "-jar", "app.jar"]
```

1.1.2 Python Backend

The Python backend Dockerfile also uses a multi-stage build. The first stage installs dependencies using Poetry and creates a virtual environment inside the project folder. The second stage copies the virtual environment and application code into a clean Python 3.11 image. A non-root user is created for security, and the service is started using Uvicorn on port 8000. A health check is also included to verify that the API is running correctly.

Listing 2: Dockerfile for Python Backend

```
    # === Build Stage ===
FROM python:3.11-slim AS builder

# Configure Poetry environment variables
ENV POETRY_VERSION=1.8.3 \
    POETRY_HOME=/opt/poetry \
    POETRY_NO_INTERACTION=1 \
    POETRY_VIRTUALENVS_IN_PROJECT=1 \
    POETRY_VIRTUALENVS_CREATE=1 \
    POETRY_CACHE_DIR=/tmp/poetry_cache

# Install required system packages
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Install Poetry
RUN curl -sSL https://install.python-poetry.org | python3 - && \
    cd /usr/local/bin && \
    ln -s /opt/poetry/bin/poetry && \
    poetry --version

# Set working directory
WORKDIR /app

# Copy dependency definitions
COPY pyproject.toml poetry.lock* ./

# Install project dependencies
RUN poetry lock --no-update 2>/dev/null || true && \
    poetry install --only main --no-root --no-directory && \
    rm -rf $POETRY_CACHE_DIR

# === Runtime Stage ===
FROM python:3.11-slim

# Configure Python runtime environment
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PATH="/app/.venv/bin:$PATH"

# Install runtime system packages
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 \
    && rm -rf /var/lib/apt/lists/*
```

```
# Set working directory
WORKDIR /app

# Copy virtual environment from builder stage
COPY --from=builder /app/.venv /app/.venv

# Copy application code
COPY . .

# Create and switch to non-root user
RUN useradd -m -u 1000 appuser && \
    chown -R appuser:appuser /app
USER appuser

# Expose service port
EXPOSE 8000

# Define health check endpoint
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost
        :8000/api/v1/health')" || exit 1

# Start the application
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

These Dockerfiles allow both services to run consistently across different environments and ensure that all dependencies and runtime configurations are properly isolated inside each container.

1.1.3 Frontend

Listing 3: Dockerfile for Frontend

```
    # === Build Stage ===
FROM node:18-alpine AS build

WORKDIR /app

# Install dependencies
COPY package*.json ./
RUN npm install

# Copy application source and generate production build
COPY . .
RUN npm run build

# === Production Stage ===
FROM nginx:alpine

# Copy Vite build output to Nginx HTML root
COPY --from=build /app/dist /usr/share/nginx/html

# Apply custom Nginx configuration
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

```
# Expose service port
EXPOSE 80

# Start Nginx server
CMD ["nginx", "-g", "daemon off;"]
```

1.2 Docker Compose Configuration

To orchestrate the application components, we created multiple docker-compose.yml and docker-stack.yml files for development and deployment. These files define how each container interacts with its database and with the reverse proxy when needed.

1.2.1 Local deployment

For local development, Docker Compose builds the images directly from the Dockerfiles and maps the service ports to the host machine.

For the Java backend, the compose file includes a MySQL database initialized with SQL scripts.

Listing 4: docker-compose.yml for Java Backend (Local)

```
version: '3.8'
services:
java-backend:
image: java-backend:latest
build:
context: .
dockerfile: Dockerfile
container_name: java-backend
ports:
- "8080:8080"
env_file:
- .env
depends_on:
- java-backend-db
networks:
- java-backend-network

java-backend-db:
image: mysql:8.0
container_name: java-backend-db
env_file:
- .env
volumes:
- ./init_db_scripts/schema_db.sql:/docker-entrypoint-initdb.d/schema_db.sql
- db_data:/var/lib/mysql
healthcheck:
test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "${MYSQL_USER}",
    "-p${MYSQL_PASSWORD}"]
interval: 10s
timeout: 5s
```

```
    retries: 5
    networks:
    - java-backend-network

    volumes:
    db_data:

    networks:
    java-backend-network:
    driver: bridge
```

For the Python backend, the compose file includes a PostgreSQL database with its initialization scripts and health checks.

Listing 5: docker-compose.yml for Python Backend (Local Development)

```
    version: '3.8'

    services:
    python-backend-db:
    image: postgres:15-alpine
    container_name: python-backend-db
    restart: unless-stopped
    env_file:
    - .env
    volumes:
    - ./init_db_scripts/schema_db.sql:/docker-entrypoint-initdb.d/schema_db.sql:ro
    - postgres_data:/var/lib/postgresql/data
    healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-postgres} -d ${POSTGRES_DB
        :-postgres}"]
    interval: 10s
    timeout: 5s
    retries: 5

    python-backend:
    build:
    context: .
    dockerfile: Dockerfile
    image: python-backend:latest
    container_name: python-backend
    restart: unless-stopped
    ports:
    - "${BACKEND_PORT:-8000}:8000"
    env_file:
    - .env
    depends_on:
    python-backend-db:
    condition: service_healthy
    command: uvicorn src.main:app --host 0.0.0.0 --port 8000 --reload

    volumes:
    postgres_data:
    driver: local
```

Both services load environment variables from .env files and wait for the databases to become healthy before starting.

1.2.2 Production deployment

For production, the docker-stack.yml files define the same services but include additional configuration such as replicas, restart policies, placement constraints, and Traefik labels. These labels allow each backend to be published through HTTPS using different hostnames and connected to a shared reverse proxy network.

Listing 6: docker-stack.yml for Java Backend

```
version: '3.8'
services:
java-backend:
image: java-backend:latest
env_file:
- .env
depends_on:
- java-backend-db
networks:
- java-backend-network
- reverse_proxy
deploy:
replicas: 1
restart_policy:
condition: on-failure
delay: 10s
max_attempts: 5
placement:
constraints:
- node.role == manager
labels:
- "traefik.enable=true"
- "traefik.docker.network=reverse_proxy"
- "traefik.http.routers.java-backend.entrypoints=websecure"
- "traefik.http.routers.java-backend.tls.certresolver=production"
- "traefik.http.routers.java-backend.rule=Host(‘authbackbs.glud.org‘)"
- "traefik.http.services.java-backend.loadbalancer.server.port=8080"
```

Listing 7: docker-stack.yml for Python Backend (Docker Swarm)

```
version: '3.8'

services:
python-backend:
image: python-backend:latest
env_file:
- .env
depends_on:
- python-backend-db
command: uvicorn src.main:app --host 0.0.0.0 --port 8000
networks:
- backend-network
```

```
      - reverse_proxy
      deploy:
      replicas: 1
      restart_policy:
      condition: on-failure
      placement:
      constraints:
      - node.role == manager
      labels:
      - "traefik.enable=true"
      - "traefik.docker.network=reverse_proxy"
      - "traefik.http.routers.python-backend.entrypoints=websecure"
      - "traefik.http.routers.python-backend.tls.certresolver=production"
      - "traefik.http.routers.python-backend.tls=true"
      - "traefik.http.routers.python-backend.rule=Host(‘coursefinderbs.glud.org‘)"
      - "traefik.http.services.python-backend.loadbalancer.server.port=8000"

      python-backend-db:
      image: postgres:15-alpine
      env_file:
      - .env
      networks:
      - backend-network
      volumes:
      - ./init_db_scripts/schema_db.sql:/docker-entrypoint-initdb.d/schema_db.sql:ro
      - postgres_data:/var/lib/postgresql/data
      healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-postgres}"]
      interval: 10s
      timeout: 5s
      retries: 5
      deploy:
      replicas: 1
      restart_policy:
      condition: on-failure
      placement:
      constraints:
      - node.role == manager

      volumes:
      postgres_data:
      driver: local

      networks:
      backend-network:
      driver: overlay
      reverse_proxy:
      external: true
```

Using Docker Compose and Docker Swarm makes it possible to run the system as a group of independent services while keeping the configuration simple, repeatable, and easy to deploy.

1.3 Makefile Usage

The backends also includes a Makefile to simplify common tasks. This file includes commands for installing dependencies, running tests, linting the code, cleaning temporary files, building Docker images, and deploying the stack in Docker Swarm. Those Makefile helps maintain a clean and consistent workflow for all team members.

Listing 8: Makefile for Java Backend

```
install:
mvn clean package

dev:
mvn clean spring-boot:run

build:
docker build -t java-backend:latest .

build-no-cache:
docker build --no-cache -t java-backend:latest .

compose-up:
docker compose up -d

compose-up-build:
docker compose up -d --build

compose-down:
docker compose down

stack-deploy:
docker stack deploy -c docker-stack.yml java-backend

stack-remove:
docker stack rm java-backend

stack-ps:
docker stack ps java-backend

stack-services:
docker stack services java-backend
```

Listing 9: Makefile for Python Backend

```
install:
poetry install

install-prod:
poetry install --only main

update:
poetry update

lock:
poetry lock
```

```
dev:
poetry run uvicorn src.main:app --reload --host 0.0.0.0 --port 8000

test:
poetry run pytest

test-cov:
poetry run pytest --cov=src --cov-report=html --cov-report=term

lint:
poetry run ruff check src

lint-fix:
poetry run ruff check --fix src

format:
poetry run ruff format src

type-check:
poetry run mypy src

clean:
find . -type d -name "__pycache__" -exec rm -rf {} +
rm -rf htmlcov .coverage

build:
docker build -t python-backend:latest .

build-no-cache:
docker build --no-cache -t python-backend:latest .

compose-up:
docker compose up -d

compose-up-build:
docker compose up -d --build

compose-down:
docker compose down

stack-deploy:
docker stack deploy -c docker-stack.yml python-backend

stack-remove:
docker stack rm python-backend

stack-ps:
docker stack ps python-backend

stack-services:
docker stack services python-backend
```

1.4 Summary

In summary, the containerization process provides a complete and isolated environment for both backends. The Dockerfiles create optimized and lightweight images, while the

Compose and Stack files configure the services, databases, networks, and the reverse proxy for different environments.

In addition to this, both backends include Makefiles that automate common tasks such as installing dependencies, running the application in development mode, building Docker images, and deploying the services with Docker Swarm. These Makefiles help maintain a consistent workflow and make the development and deployment process faster and easier. Overall, these containers make the project simple to run, test, and deploy in both development and production environments.

2 Frontend Acceptance Tests with Cucumber

2.1 Overview

For this project, we used Cucumber to create frontend acceptance tests based on the main user stories. The goal of these tests is to verify the behavior of the user interface and interactions from the perspective of the final user. By focusing on frontend tests, we ensure that the system responds correctly to user actions such as form submissions, navigation between tabs, and data creation flows.
Currently, we implemented a single acceptance test for the *Create Professor* feature, based on the `createProfessor.feature` file. This test validates that a system administrator can successfully add new teachers and that the platform enforces proper data entry and duplication rules.

2.2 Feature File: Create Professor

The `createProfessor.feature` file describes the following scenarios:

- **Access the professor registration form:** Ensures that the administrator can navigate to the registration form from the main panel.
- **Register a teacher with valid data:** Validates that filling out all required fields with correct information allows successful registration, and the new teacher appears immediately in the list.
- **Validate required fields:** Checks that leaving required fields empty triggers validation messages.
- **Prevent duplicate teacher registration:** Confirms that attempting to register a teacher with an existing email or ID is blocked and shows an error.

Each scenario follows the standard *Given-When-Then* structure for clarity and maintainability.

2.3 Step Definitions

Step definitions connect the Gherkin steps with executable frontend code, including:

- Navigating the administrator panel to reach the registration form.

- Filling out input fields, submitting the form, and verifying messages.
- Ensuring new entries appear in lists dynamically without page reload.
- Detecting validation and duplication errors.

This structure allows easy addition of new frontend acceptance tests as the platform grows.

2.4 Execution and Test Results

The *Create Professor* test was executed in the frontend testing environment. The Cucumber report generated shows which steps passed and which encountered issues. The results provide a clear overview of the test status, highlighting both successes and known errors.

Figure 1: Cucumber acceptance test results for the "Create Professor" feature in the frontend.

Even with some steps failing due to minor frontend issues, the test confirms that the main workflow of creating a professor works correctly and establishes a baseline for future frontend validations.

2.5 Summary

Frontend acceptance tests with Cucumber allow us to validate user interactions and ensure the core flows behave as expected. The *Create Professor* test demonstrates this approach, providing confidence that the teacher registration feature functions according to requirements.

3 GitHub Actions Workflow

The project uses GitHub Actions to implement a continuous integration (CI) pipeline that validates the three services in the system—frontend, Java backend, and Python backend—every time new changes are pushed or merged. The goal is to automate linting, testing, and building across all components, ensuring that the system remains stable and that regressions are detected early. This setup fits naturally with the microservice-oriented structure of the workshop, where each service evolves independently but must meet a shared quality bar.

3.1 Overview

For this project, we created a GitHub Actions workflow to automate important tasks such as running tests and building Docker images. The goal of this workflow is to support

continuous integration and continuous delivery (CI/CD). Each time we push new code to the repository, the workflow runs automatically and checks that the system continues to work correctly.
Using a multi-workflow CI architecture allows each service to be tested in isolation using the runtime and database it requires. The Java backend runs against MySQL, the Python backend against PostgreSQL, and the frontend uses Node.js tooling—each with its own optimized environment. This separation speeds up execution, avoids cross-service contamination, and mirrors real production pipelines where microservices are validated independently before integration.
Additionally, keeping a unified pipeline (workshop4-ci.yaml) ensures that the whole system can be validated together whenever changes affect the full codebase. The repository contains four workflow files:

- **workshop4-ci.yaml:** Unified pipeline that validates the full system.
- **frontend-ci.yaml:** CI pipeline for the frontend service.
- **java-backend.yaml:** CI pipeline for the Java backend.
- **python-backend.yaml:** CI pipeline for the Python backend.

This structure makes the CI process modular, scalable, and aligned with real-world production pipelines.

3.2 Workflow Structure

The unified CI workflow (`workshop4-ci.yaml`) runs when changes are made that affect the entire repository. Its purpose is to verify the integrity of the whole system and catch issues that appear when services interact.
The main responsibilities of the unified workflow are:

1. Check out the repository and prepare the working environment.
2. Install dependencies for all services.
3. Run the full test suite across frontend, Java backend, and Python backend.
4. Build Docker images for the backend services.

By validating all components together, this workflow provides confidence that system-wide changes do not break integration.
Each service includes its own isolated workflow that runs independently of the others. This separation improves performance, reduces unnecessary computations, and mirrors the autonomy of microservice deployments.

3.2.1 Frontend Workflow

The frontend workflow (`frontend-ci.yaml`) focuses on the Node.js environment. Its main steps are:

1. Install Node.js dependencies via `npm`.
2. Run linting and static analysis tools.
3. Build the production-ready Vite bundle.

3.2.2 Java Backend Workflow

The Java backend workflow (`java-backend.yaml`) uses Maven and runs against a MySQL service for integration tests.

1. Configure the Java 21 (Corretto) environment.
2. Restore Maven dependencies to speed up builds.
3. Run unit and integration tests.
4. Package the application into a JAR file.
5. Build the Docker image using the project's Dockerfile.

3.2.3 Python Backend Workflow

The Python backend workflow (`python-backend.yaml`) creates an isolated Python environment and runs against PostgreSQL.

1. Set up Python 3.11 and install dependencies via Poetry.
2. Run unit tests and API-level tests.
3. Validate the FastAPI application.
4. Build the Docker image defined for the Python backend.

Each backend workflow ensures that its service can be independently built, tested, and containerized.

3.3 Automatic Testing

One important part of the workflow is the automatic test execution. When the workflow runs, it executes the corresponding test suites. If any test fails, the workflow stops and reports the error. This helps identify problems early and ensures code quality.
The results show whether all tests passed and allow developers to correct errors before merging new changes.

3.4 Docker Image Build

The workflow also builds the Docker images for the backends. This guarantees that the Dockerfiles are correct and that the application can be containerized without errors. The images can later be published or used in a deployment environment.
Building the images automatically helps maintain consistency between development and production environments. A key part of the CI pipeline is the execution of automated tests. Whenever a workflow runs:

- All unit and integration tests are executed.
- Any failure immediately stops the workflow.
- Clear logs and error messages are reported in GitHub Actions.

Additionally, each backend workflow builds its corresponding Docker image:

- Ensuring Dockerfiles remain functional.
- Verifying that the services can be containerized at any time.
- Guaranteeing consistency between development and production environments.

3.5 Workflow Results

After each execution, GitHub Actions provides detailed logs and a summarized status report that indicate whether every job in the pipeline completed successfully. These results help validate the stability of the system and ensure that all services continue to work as expected after each change.

3.5.1 General Overview

When the workflow finishes, GitHub Actions displays a high-level summary showing the execution status and duration of each job. This global view allows developers to quickly confirm that the CI pipeline completed without errors. For example, a successful run of the unified workflow may produce a summary like:

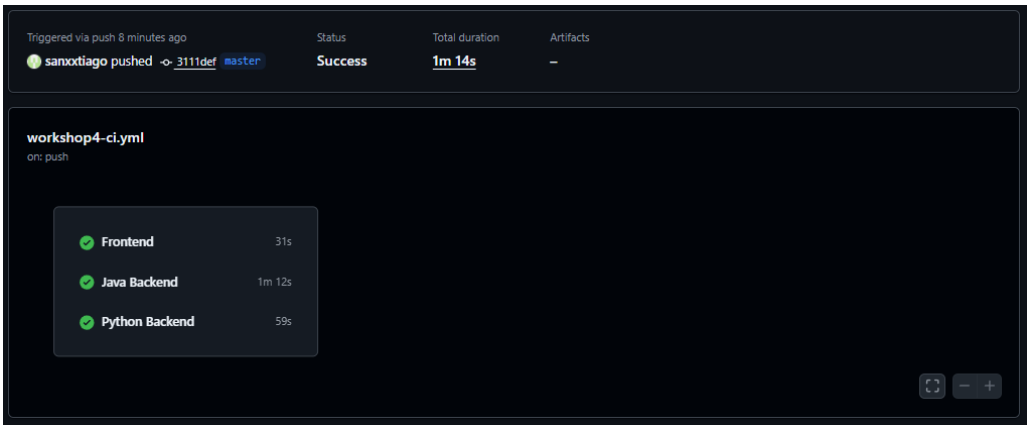

Figure 2: Summary of a successful run of the unified CI workflow.

This overview makes it easy to detect which component failed or slowed down, helping maintain smooth development across the three services.

3.5.2 Service-Specific Results

Each job also includes detailed logs covering all steps performed during the execution. These logs show:

- the installation of dependencies,
- test execution and pass/fail status,
- Docker image build output,
- warnings or errors raised during the workflow,

- and the final success or failure of the job.

This per-service granularity lets developers investigate issues without affecting the other components, which is particularly useful in a microservice architecture where each service evolves independently.

3.5.3 Interpreting Success and Failure

A workflow run is considered fully successful only when:

1. all tests pass,
2. all Docker images build correctly,
3. and all jobs complete without warnings that block execution.

If any job fails, GitHub Actions marks the entire workflow as failed and highlights the problematic job. This immediate feedback loop ensures that regressions are identified early and prevents unstable code from being merged.

3.6 Summary

The GitHub Actions setup provides a robust, automated pipeline for validating the entire system. By combining a unified workflow with service-specific pipelines, the project benefits from fast feedback, cleaner code, and reliable CI/CD practices. This design reduces manual intervention, improves code quality, and ensures that every commit is thoroughly tested.

4 Conclusion

This workshop demonstrated the implementation of modern software engineering practices through containerization, frontend acceptance testing, and continuous integration. Each of the objectives set for this project was addressed systematically:

- **Docker Containerization:** All components of the system—including the Java backend, Python backend, and frontend—were successfully containerized. The use of multi-stage Dockerfiles ensured lightweight and optimized images, while Docker Compose and Docker Swarm orchestrations allowed seamless deployment in both development and production environments. The containers guarantee environment consistency, reduce dependency conflicts, and simplify setup for developers and testers.
- **Acceptance Testing with Cucumber:** Frontend acceptance tests were implemented using Cucumber to validate critical user stories. The *Create Professor* feature illustrated how administrators interact with the system, covering scenarios such as form access, valid teacher registration, required field validation, and duplicate prevention. Although some steps failed, the test provided valuable feedback on UI behavior and set a foundation for future automated frontend validations.

- **CI/CD Pipeline:** A modular CI/CD pipeline using GitHub Actions was created for the entire project. Each service—frontend, Java backend, and Python backend—has dedicated workflows for installing dependencies, running tests, and building Docker images. A unified workflow was also implemented to ensure system-wide integrity. This setup enforces consistent quality, detects regressions early, and streamlines deployment processes.

Overall, this workshop provided a practical demonstration of integrating containerization, automated testing, and CI/CD in a microservices environment. By combining these practices, the project achieved reproducible deployments, reliable validation of user-facing features, and automated system verification. The outcomes highlight the importance of these methodologies in maintaining code quality, reducing manual effort, and ensuring smooth collaboration across development teams.